Partie 3 : Agir et Mémoriser — Exécution Sécurisée, Vérification Réseau et Traçabilité

RoamPredict · rapport technique de la partie 3 · Adil · tests sur le terrain du 5 au 7 octobre 2026 · 264 tests automatiques

Les parties 1 et 2 de RoamPredict observent le signal, prévoient la perte de la liaison et choisissent la meilleure borne de remplacement. La partie 3 intervient au moment où cette décision est prise : elle **exécute le basculement** de manière sûre, **vérifie** que la connexion internet est réellement rétablie, puis **mesure et enregistre** chaque coupure et chaque basculement. Elle regroupe sept fichiers : `executor.py`, `l3.py`, `win32_structs.py`, `probe.py`, `latency.py`, `store.py` et `telemetry.py`.

Ce rapport suit la chaîne dans l'ordre où elle s'exécute : agir, vérifier, communiquer avec Windows, mesurer, mémoriser, tracer. Il se termine par la campagne de marches sur le campus, qui montre comment les données enregistrées par la partie 3 ont permis d'améliorer le code.

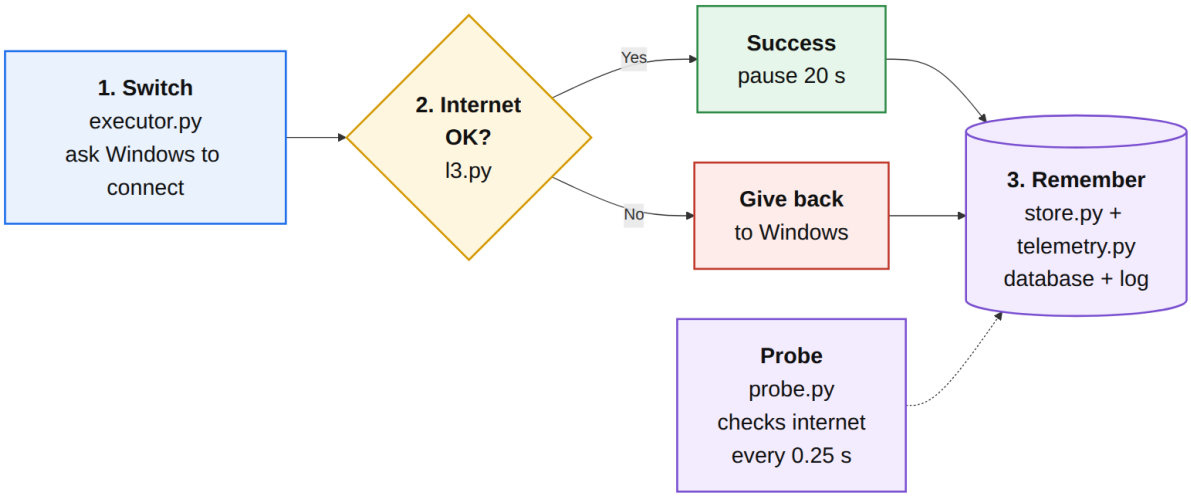


Figure 1. La partie 3 en une image : basculer (Switch), vérifier internet, succès ou retour à Windows (Give back), puis mémoriser (Remember). La sonde (Probe) mesure en continu.

1. Agir (Exécution Sécurisée du Basculement — executor.py)

Pourquoi une procédure stricte ? Windows ne propose aucune commande publique « change de borne maintenant ». Le seul levier disponible est la fonction `WlanConnect`, qui signifie : « connecte-toi avec ce profil enregistré, et de préférence à cette borne précise (BSSID) ». En pratique, Windows coupe d'abord l'ancienne liaison puis se reconnecte, ce qui impose une courte coupure à chaque basculement (entre 2 et 7 s mesurées sur le campus). Un basculement mal choisi ou mal contrôlé coûte donc plus cher que de ne rien faire. La classe `RoamExecutor` applique pour cette raison une séquence en plusieurs étapes, où chaque étape peut arrêter la procédure.

Étape 1 : Vérification de la cible sur des données fraîches

Les mesures utilisées pour prendre la décision peuvent dater de plusieurs secondes. Avant d'agir, la méthode `verify()` relit la liste des bornes visibles. Si la borne candidate n'a pas été entendue depuis plus de **3 s** (`fresh_scan_max_age_s`), elle demande un nouveau scan et l'attend au maximum **4,5 s**. Trois conditions doivent alors être réunies :

- **Visibilité :** la borne candidate apparaît dans la liste rafraîchie, avec le même SSID que le réseau actuel (on ne quitte jamais le réseau autorisé).

- **Stabilité de la liaison :** le portable est toujours connecté à la même borne. Si Windows a basculé entre-temps de lui-même, la décision n'a plus de sens et elle est abandonnée.
- **Gain minimal :** la borne candidate doit dépasser la borne actuelle d'au moins **+8 dB** (+3 dB seulement si internet est déjà coupé, car dans ce cas rester sur place n'a plus aucun intérêt). Le gain est calculé ainsi : gain = $\text{RSSI}_{\text{candidat}} - \text{pénalité}_{\text{2,4 GHz}} - \text{RSSI}_{\text{actuel}}$.

Si l'une de ces conditions échoue, le résultat est `NOT_VERIFIED` et **aucun basculement n'a lieu**. C'est l'application directe de la règle du cahier des charges : ne jamais couper la connexion sans qu'une meilleure borne soit vérifiée.

Étape 2 : Demande de connexion par nom de profil

L'exécuteur appelle `connect_bssid()`, qui remplit la structure `WLAN_CONNECTION_PARAMETERS` avec **uniquement le nom du profil Windows** (`params.strProfile = profile`) et le BSSID souhaité. Le mot de passe ou le certificat restent stockés dans Windows : le programme Python ne les lit, ne les demande et ne les enregistre jamais. Pendant la connexion, les notifications de Windows permettent de chronométrer chaque phase : association radio, authentification 802.1X, puis connexion établie. Le délai maximal est de **12 s**.

Étape 3 : Traitement des échecs de connexion

Deux régimes d'échec sont distingués, car ils n'appellent pas la même réaction :

1. *L'authentification bloquée :* si la phase 802.1X ne progresse plus pendant **6 s** (`auth_stall_s`), ou si Windows renvoie un code d'erreur de la famille authentification (entre `0x40000` et `0x5FFFF`), une fenêtre de connexion pourrait apparaître à l'écran. Le programme s'arrête immédiatement, le signale (`login_prompt_suspected = True`) et laisse Windows gérer seul. Le moteur déclenche alors l'arrêt d'urgence.
2. *L'échec ordinaire :* la borne n'a pas répondu ou la connexion a été refusée. On laisse d'abord **5 s** à Windows pour se reconnecter de lui-même. S'il n'y parvient pas, la méthode `_hand_back()` redemande une connexion avec le nom du profil, **sans imposer de borne**, ce qui rend le choix entièrement à Windows.

Étapes 4 à 7 : Contrôle, nettoyage et mesure

- **Contrôle du BSSID :** on vérifie que le pilote s'est bien connecté à la borne demandée et non à une autre. Un écart est signalé dans le résultat.
- **Contrôle de la couche IP :** confié à `l3.py` (section 2). Si le portable est connecté au Wi-Fi mais sans adresse ni passerelle, l'exécuteur rend la main à Windows. Cette règle a été ajoutée après une marche où un basculement réussi au niveau Wi-Fi n'a jamais obtenu d'adresse IP.
- **Nettoyage :** `SocketRegistry.reset_all()` ferme les connexions ouvertes par RoamPredict sur l'ancienne borne, pour que la sonde reparte proprement. Les autres programmes ne sont pas touchés.
- **Mesure :** tous les temps (connexion, authentification, obtention de l'IP) sont rangés dans un objet `RoamOutcome`, converti ensuite en `SwitchRecord` pour la base de données.

Après le basculement : le résultat est transmis par le moteur aux règles de sécurité (`safety.py`). Un succès impose une pause de **20 s** avant tout nouveau basculement, un échec une pause de **60 s**. Après deux échecs consécutifs, ou dès qu'une fenêtre de connexion est suspectée, l'arrêt d'urgence bloque toute action jusqu'à la commande `roampredict unkill`.

2. Vérifier (Contrôle de la Couche Réseau — l3.py)

Pourquoi vérifier au-delà du Wi-Fi ? Être associé à une borne ne garantit pas qu'internet fonctionne. Après un basculement, trois éléments doivent encore être en place : une **adresse IP** valide, une **passerelle** (la porte de sortie vers internet) et une **table ARP** correcte (la correspondance entre l'adresse IP de la passerelle et son adresse physique). Le fichier `l3.py` lit ces informations directement par les API de Windows, plutôt que d'analyser le texte affiché par des commandes, ce qui le rend indépendant de la langue du système.

Les outils utilisés :

- `GetAdaptersAddresses` : lit l'adresse IPv4, le préfixe du sous-réseau, la passerelle et le serveur DHCP de l'adaptateur Wi-Fi. Aucun droit administrateur n'est nécessaire.
- `IcmpSendEcho` : envoie un ping vers la passerelle, avec un délai de 500 ms par essai.
- `FlushIpNetTable2` : vide le cache ARP de l'interface Wi-Fi. Nécessite les droits administrateur.
- `ipconfig /renew` : redemande une adresse au serveur DHCP. Lancé avec une liste d'arguments et le chemin absolu `%SystemRoot%\System32\ipconfig.exe`, jamais via un shell (`shell=True` est interdit), pour qu'aucun autre programme ne puisse être substitué.

La logique de décision : l'exécuteur prend un « instantané » de la configuration IP *avant* le basculement, puis attend jusqu'à **4 s** un instantané utilisable *après*. La comparaison des deux détermine l'action :

1. *Même sous-réseau, même passerelle :* l'adresse reste valable ; seul le cache ARP risque d'être périmé, il est donc vidé.
2. *Réseau changé ou aucune adresse :* un renouvellement DHCP est demandé.
3. *Adresse en 169.254.x.x :* c'est une adresse « de secours » que Windows s'attribue quand le serveur DHCP n'a pas répondu. Elle ne permet pas d'aller sur internet et elle est traitée comme une absence d'adresse.

Adaptation au campus : le réseau du campus bloque le ping vers la passerelle. Pour éviter de conclure à tort que la connexion est morte, l'exécuteur teste le ping *avant* le basculement, pendant que l'ancienne liaison fonctionne encore. Si la passerelle ne répondait déjà pas, l'absence de réponse après le basculement ne prouve rien, et le contrôle porte alors sur la validité de l'adresse IP.

3. Communiquer avec Windows (Interface Native Win32 — win32_structs.py)

Le problème : les bibliothèques de Windows qui gèrent le Wi-Fi (`wlanapi.dll`) et le réseau IP (`iphlpapi.dll`) sont écrites en langage C. Leurs fonctions attendent des données rangées en mémoire octet par octet, dans un format très précis. Python ne peut pas les appeler directement.

La solution : le module standard `ctypes` permet de décrire ces formats en Python. Le fichier `win32_structs.py` joue le rôle de **traducteur** : il reconstruit chaque structure C champ par champ, par exemple :

- `WLAN_BSS_ENTRY` : une borne vue lors d'un scan (BSSID, puissance du signal, fréquence, âge).
- `WLAN_CONNECTION_PARAMETERS` : la « fiche » remplie pour demander une connexion (nom du profil, liste des BSSID souhaités).
- `IP_ADAPTER_ADDRESSES_LH` : les adresses IP et la passerelle de l'adaptateur.
- `ICMP_ECHO_REPLY` : la réponse à un ping.

Le fichier ne communique pas lui-même avec Windows : ce sont `windows_wlan.py` et `l3.py` qui appellent les fonctions, en utilisant ces structures. La chaîne complète est : programme Python → `wlanapi.dll` → service Wi-Fi de Windows → pilote → carte Wi-Fi.

La fiabilité : une structure mal décrite corromprait la mémoire sans message d'erreur clair. Trois précautions l'empêchent : seuls des entiers de taille fixe sont utilisés, de sorte que chaque structure a exactement la disposition de Windows 64 bits ; 30 tests vérifient la taille de chaque structure et la position de chaque champ ; enfin, chaque fonction appelée a ses types d'arguments déclarés, ce qui transforme un mauvais appel en erreur Python au lieu d'un plantage. Par sécurité, certaines fonctions ne sont **jamais** utilisées : `WlanGetProfile`, qui pourrait exposer le contenu d'un profil, et `WlanDisconnect`, car le programme ne coupe jamais la connexion lui-même.

4. Mesurer (Sonde de Connectivité et Détection des Coupures — probe.py, latency.py)

Pourquoi une sonde ? Le Wi-Fi peut être connecté alors qu'internet ne répond pas encore (authentification, DHCP ou ARP en cours). Ce qui compte pour l'utilisateur, c'est le temps réel sans internet. Avec l'option `run --probe www.google.com`, une sonde « frappe à la porte » de Google **toutes les 0,25 s** (4 fois par seconde) en ouvrant une connexion TCP sur le port 443, celui qu'utilise un navigateur pour les sites en https.

Les choix de mesure :

- **TCP plutôt que ping :** le campus bloque le ping et certaines adresses publiques comme 1.1.1.1. Le port 443 de Google, lui, est toujours ouvert, sinon aucun site ne serait accessible.
- **Résolution unique du nom :** l'adresse de www.google.com n'est cherchée qu'une seule fois, au démarrage. Une lenteur du DNS (l'annuaire d'internet) n'est donc jamais comptée comme une coupure.
- **Exécution en parallèle :** la sonde tourne dans un *thread* d'arrière-plan, pendant que le programme principal surveille le Wi-Fi. Elle se contente de déposer ses événements dans une file, et toutes les décisions restent prises par le thread principal.

Les quatre classes et leur rôle :

1. `TcpPinger` *(latency.py) :* réalise une tentative de connexion chronométrée (délai maximal 1 s). En cas d'échec, il note la raison (délai dépassé, connexion refusée, DNS…) dans un `Counter`, un dictionnaire qui compte, par exemple `{"timeout": 12}`.
2. `OutageTracker` *(probe.py) :* transforme la suite des résultats en coupures. Une coupure commence au premier essai sans réponse et n'est confirmée qu'après **2 échecs consécutifs** ; elle se termine à la première réponse. Sa durée vaut : durée = $t_{\text{première réponse}} - t_{\text{premier échec}}$.
3. `ConnectivityProbe` *(probe.py) :* fait tourner la sonde dans le thread et prévient le moteur par deux événements : `NET_DOWN` (internet coupé) et `NET_UP` (internet revenu).
4. `SocketRegistry` *(latency.py) :* garde la liste des connexions ouvertes par RoamPredict, pour pouvoir les fermer toutes après un basculement.

Pourquoi deux échecs et pas un ? Sur un réseau chargé, un paquet isolé se perd parfois même quand internet fonctionne. Avec un seul échec, on enregistrerait de fausses coupures. Deux échecs consécutifs (soit environ 0,5 s) confirment une vraie coupure, tout en gardant une précision d'un quart de seconde. Enfin, une coupure encore en cours au moment où le programme s'arrête est enregistrée jusqu'à cet instant : arrêter le programme ne fait jamais disparaître une coupure des statistiques.

5. Mémoriser (Persistance des Données — store.py)

Pourquoi une base de données ? Pour comparer un trajet avec et sans RoamPredict, il faut garder une trace fiable de chaque marche, même après avoir fermé le programme. Toutes les données sont enregistrées dans un seul fichier SQLite, `%LOCALAPPDATA%\RoamPredict\roampredict.db`, qui reste sur le portable.

L'accès contrôlé : la base n'est jamais manipulée directement. Tout passe par la classe `TopologyStore`, selon le patron de conception « dépôt » : elle cache le langage SQL au reste du programme et renvoie chaque ligne sous forme de *dataclass* Python. Les requêtes sont paramétrées (aucune valeur n'est collée dans le texte SQL) et le fichier fonctionne en mode `WAL`, qui permet de lire pendant une écriture.

Ce qui est enregistré :

- **Les coupures (`outages`) :** début, durée, couche concernée (Wi-Fi ou internet), mode (dry-run ou actif) et dernier signal mesuré. C'est cette table qui permet de dire « avec » ou « sans » RoamPredict.
- **Les basculements (`switch_events`) :** ceux de RoamPredict comme ceux de Windows, avec leurs durées, leur succès, la borne demandée et la borne obtenue.
- **Les associations et les déplacements appris (`associations`, `transitions`) :** chaque connexion à une borne et chaque passage d'une borne A à une borne B, avec un compteur. La partie 2 s'en sert pour apprendre les trajets habituels.
- **Les bornes connues (`bss`) :** chaque borne déjà vue, avec son canal, sa bande et ses options.

La validation à l'écriture : une coupure avec une couche ou un mode inconnu, ou une durée négative, est refusée. La commande `roampredict report --timeline` relit ensuite ces tables pour afficher, dans l'ordre, toutes les coupures et tous les basculements d'une marche.

6. Tracer (Journal d'Événements — telemetry.py)

Le principe de la boîte noire : chaque événement du programme est écrit sous forme d'une ligne JSON dans `%LOCALAPPDATA%\RoamPredict\logs\roampredict.jsonl`. Les plus importants sont aussi affichés en lignes courtes à l'écran (`START`, `STATE`, `DECISION`, `SWITCH_RESULT`, `NET_DOWN`, `STOP`…). Le fichier n'est jamais effacé, seulement complété : il conserve l'historique de toutes les marches. Exemple réel d'une ligne :

```
{"ts": "2026-10-07T16:44:16+0000", "kind": "switch_result",
 "msg": "OK: switched [connect 1914 ms (auth 773 ms), IP ready after 13 ms]",
 "success": true, "total_ms": 1914.0, ...}
```

- **Protection de la vie privée :** avant chaque écriture, la fonction `scrub()` parcourt l'événement et remplace par « [removed] » tout champ dont le nom ressemble à un mot de passe, un identifiant, un jeton ou un certificat. C'est une deuxième ligne de défense, car le programme ne possède de toute façon aucune de ces informations. Aucune donnée de navigation n'est enregistrée.
- **Sûreté entre threads :** le thread de la sonde et le thread principal peuvent écrire en même temps ; un verrou garantit que deux lignes ne se mélangent jamais.
- **Compatibilité :** l'affichage console reste en caractères ASCII simples, car une console Windows redirigée vers un fichier ne supporte pas toujours les caractères spéciaux.

7. Structures de Données (Listes et Dictionnaires)

La partie 3 s'appuie sur les structures de base de Python, chacune choisie pour un besoin précis :

- **Dictionnaire de listes** (`engine.py:200`) : `session_outages = {"wifi": [], "internet": []}` range les durées de coupure par couche. À la fin de la marche, `len`, `sum` et `max` donnent directement le nombre, le total et la plus longue coupure de la ligne STOP.
- **Liste ordonnée** (`probe.py:71`) : `self.outages: list[Outage] = []` conserve les coupures dans l'ordre chronologique où elles se produisent.
- **Compteur** (`latency.py:114`) : `Counter` est un dictionnaire spécialisé qui compte les raisons des sondes perdues ; c'est lui qui a révélé que l'adresse 1.1.1.1 était bloquée sur le campus.
- **Regroupement** (`store.py:484`) : `groups.setdefault(key, []).append(rec)` regroupe les basculements par type pour en calculer les durées médianes.
- **Dictionnaire d'événement** (`telemetry.py`) : chaque événement est d'abord un dictionnaire `{"ts": ..., "kind": ...}`, transformé ensuite en une ligne JSON.

8. Validation sur le Terrain (Campagne de Marches et Amélioration du Code)

Pourquoi aller sur le terrain ? Un programme qui agit sur une vraie connexion ne peut pas être validé uniquement en théorie : les pilotes, le réseau du campus et Windows lui-même ont des comportements qu'aucune simulation ne reproduit. Nous avons donc marché sur le campus (réseau « Campus Connecte », 25 bornes) avec RoamPredict en fonctionnement. C'est la partie 3 qui a rendu cette campagne possible : la sonde mesure chaque coupure au quart de seconde, la base enregistre chaque coupure et chaque basculement, et le journal garde le détail de chaque décision.

8.1 Méthode

- **Trajet et durée identiques :** une boucle à pied (bibliothèque → buvette, puis l'atelier), le plus souvent de 6 minutes (`--duration 360`).
- **Deux modes comparés :** *sans* RoamPredict (mode dry-run : Windows gère seul, RoamPredict observe et mesure) et *avec* RoamPredict (option `--active`).
- **Deux portables**, chacun avec son propre profil enregistré, marchant côte à côte.
- **Indicateur principal :** le temps sans internet, en secondes et en pourcentage de la marche.

8.2 Les résultats

Une dizaine de marches ont été réalisées sur trois jours. Sans RoamPredict, la part de la marche passée sans internet a varié de **32 % à 92 %** selon le moment et le portable, ce qui montre l'ampleur du problème et la forte variabilité de Windows seul. Les coupures gérées par Windows duraient typiquement **environ 36 s** (médiane de 10 coupures le premier jour), et l'on observe sur le campus qu'internet est déjà coupé depuis 5 à 12 s lorsque Windows commence à réagir.

Sur le même trajet de l'atelier, le 6 octobre, une coupure gérée par Windows a duré **38,4 s**, alors que la coupure accompagnant le basculement réalisé par RoamPredict n'a duré que **6,6 s**. Sur le second portable, un basculement de RoamPredict a donné une coupure de 19 s.

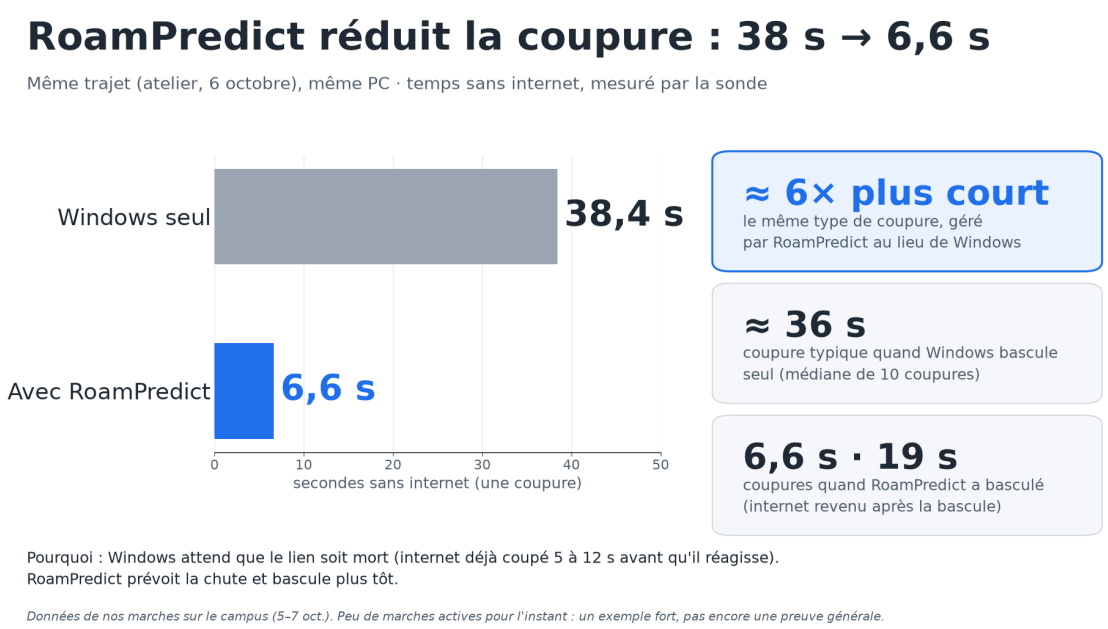


Figure 2. Même trajet, même portable : 38,4 s de coupure avec Windows seul, 6,6 s avec RoamPredict.

8.3 Ce que les données ont révélé, et comment le code a été amélioré

Chaque problème ci-dessous a été découvert dans la base ou le journal enregistrés par la partie 3, puis corrigé dans le code, protégé par un test automatique et publié sous forme d'un commit :

1. *Signal absurde du pilote :* la carte Intel AX201 renvoyait parfois +27 dBm, une valeur physiquement impossible. Elle est désormais rejetée au profit de la qualité du lien convertie (RSSI = qualité / 2 − 100).
2. *Ping bloqué vers la passerelle :* après un basculement, le contrôle porte sur l'adresse IP lorsque la passerelle ne répondait déjà pas au ping.
3. *Erreur « scan déjà en cours » (code 170) :* traitée comme une situation normale au lieu d'une erreur.
4. *Toutes les sondes vers 1.1.1.1 perdues :* grâce au compteur des raisons d'échec, l'adresse a été identifiée comme bloquée ; la sonde vise désormais www.google.com sur le port 443.
5. *Coupures internet invisibles :* seules les coupures Wi-Fi étaient mesurées ; chaque coupure internet est maintenant chronométrée et enregistrée (option `--probe`).
6. *Deux versions du code mélangées :* chaque marche affiche désormais la version exacte du code utilisée.
7. *Fausses chutes après chaque reconnexion :* la pente du signal est ignorée pendant 9 s après une reconnexion.
8. *Plantage sur un grand code d'erreur Windows :* le formatage a été corrigé, et l'apprentissage des trajets a été limité à un seul réseau.
9. *Bornes 2,4 GHz trompeuses :* elles paraissaient plus fortes mais étaient plus lentes ; un handicap de 7 dB leur est maintenant appliqué.
10. *Signal figé pendant une coupure :* une marche active s'est déroulée sans aucun basculement, car la mesure du signal restait bloquée. Nouvelle règle : si internet est mort, on se fie au dernier scan et on accepte la borne la plus forte avec seulement +3 dB.
11. *Connexion Wi-Fi sans adresse IP (169.254.x) :* l'exécuteur rend désormais la main à Windows par une reconnexion sur le nom du profil.

Bilan : onze problèmes réels, tous invisibles avant d'aller sur le terrain, ont été trouvés et corrigés en trois jours. Le code actuel est nettement plus fiable que la version du 4 octobre, et chaque correction est protégée par un test : un problème corrigé ne peut pas revenir sans que les 264 tests le signalent. Sans la partie 3, il n'y aurait pas eu de données ; sans ces données, aucune de ces améliorations n'aurait été possible.

9. Limites et Conclusion

Les limites connues :

- **Coût du basculement :** `WlanConnect` coupe d'abord l'ancienne liaison, car Windows ne permet pas de se connecter à la nouvelle borne avant de quitter l'ancienne. Chaque basculement coûte donc quelques secondes.
- **Droits administrateur :** sans eux, le vidage ARP et le renouvellement DHCP sont sautés.
- **Authentification :** la partie 3 ne peut pas aider lorsque l'authentification 802.1X de Windows échoue de façon répétée ; elle s'arrête alors volontairement.
- **Preuve statistique :** il y a encore trop peu de marches actives pour affirmer que RoamPredict est meilleur dans tous les cas. Deux portables au même endroit se comportent aussi très différemment : la comparaison doit se faire sur le même portable et le même trajet.

Conclusion : la partie 3 transforme une décision en action sûre, et chaque marche en données exploitables. Chaque basculement est vérifié avant d'agir, n'utilise que le nom du profil enregistré, est contrôlé jusqu'à la couche IP et rend la main à Windows dès qu'un problème survient. Chaque coupure est mesurée au quart de seconde et enregistrée avec son mode. Ces données ont été le moteur de l'amélioration du projet ; la prochaine étape consiste à répéter la même méthode sur le même trajet, avec davantage de marches, pour mesurer précisément le gain de RoamPredict face à Windows seul.